

Onboarding Assignment

World Cup 2026 Predictions App

Welcome to the team! This assignment is a self-contained project that walks you through how we build with **Flutter** and how our **MVVM** architecture works in practice. By the end you'll have a working cross-platform app and a solid grasp of our conventions.

Theme	FIFA World Cup 2026 (opens June 11, 2026 — Mexico vs South Africa)
Platforms	Android, iOS, Web (consistent UX across all three)
Stack	Flutter + our internal MVVM package (frontend-v2)
Est. duration	Up to 1 week (it's a learning project, not a race)

The goals of this project

- Get comfortable building a Flutter app that runs on web, Android, and iOS from one codebase.
- Learn our MVVM pattern hands-on by consuming the shared mvvm_remepy package.
- Practice working against real external APIs, plus building mock/admin tooling for testing.
- Get into the habit of writing unit tests and committing your work to git as you go.

1. Architecture & the MVVM package

This is the part that matters most for your onboarding. The app must be built on top of our shared MVVM package — do **not** roll your own state-management approach.

Where it lives

The package is in our frontend-v2 repository at:

```
packages/mvvm_remepy
```

Add it to your project as a local path (or git) dependency in `pubspec.yaml` rather than copy-pasting the code, e.g.:

```
dependencies:
  mvvm_remepy:
    path: ../frontend-v2/packages/mvvm_remepy
```

What we expect to see

- Every screen has a **ViewModel** that extends the base class from `view_model.dart`. The View (widget) holds no business logic — it only renders state and forwards user intents to the ViewModel.
- Models are plain data classes (games, users, guesses, scores). Keep API/serialization concerns out of the View.
- A thin service/repository layer sits between ViewModels and the data sources (API + local mock store). ViewModels talk to repositories, never directly to `http`.
- Read the package source first. If something about the base ViewModel's lifecycle or notification mechanism is unclear, that's a good question to bring to your buddy — understanding it is part of the assignment.

Suggested folder layout: `lib/models`, `lib/view_models`, `lib/views`, `lib/services` (repositories + API client + mock store), `lib/widgets` (shared UI like the flag/game row).

2. Functional requirements

Auth

- Users sign up and sign in with **email** (email + password is fine). Persist the session so a returning user stays logged in.
- You can use **Firebase Authentication** for this — it keeps things simple. Auth doesn't need to be complex; basic email sign up / sign in is all we're after.

Games

- **Upcoming games** screen — list the next fixtures pulled from a real data source (see section 3).
- **Results** screen — show finished games with their final scores.
- Each game/guess row shows the **flags of both teams** for a nicer look.

Predictions & scoring

- Users guess the outcome of each upcoming game: **Team A wins**, **Team B wins**, or **draw**. Guesses lock once the game starts.
- **+1 point** for each correctly guessed game.
- **+2 bonus points** for getting a full relevant set right (treat this as: all games on the same day correct). Make the 'set' definition a single, clearly-documented function so we can tweak the rule later.

Leaderboard

- A ranking screen showing the **top 10 users** by total points. If the logged-in user is outside the top 10, also show their own rank and points (e.g. as an extra row pinned below the table) so they always know where they stand.

Bonus — "new results" popup

- When a user logs in and there are **finished-game results they haven't seen yet** (e.g. results settled since their last visit), show a **popup dialog** summarising those updates. Track per-user which results have been acknowledged so the popup only appears for genuinely new ones, and not again once dismissed.

Scoring automation

- After a game finishes, an **automatic service** recomputes results, updates user scores, and refreshes the leaderboard. No live/in-play updates needed — just settle everything once a match ends.
- Implement this as a background service / periodic check (or a hook triggered by the admin/mock panel described in section 4) so it works the same in testing and in 'real' mode.

3. Data sources (APIs)

You need fixtures and results. Two solid options — start with the free one and only reach for the keyed API if you want live data:

Recommended (free, no key)

- **openfootball/worldcup.json** — public-domain JSON with the full 2026 schedule and results, including team names, dates, groups and scores. No API key. Great for both upcoming fixtures and finished results. (github.com/openfootball/worldcup.json — the 2026/worldcup.json file.)

Alternative (live data, free tier with key)

- **API-Football** (api-football.com) — covers WC 2026 via `league=1&season=2026`, fixtures and standings, updated frequently. Requires a free account/key. Use this if you want genuinely live results rather than the static JSON.

Wrap whichever you choose behind a repository interface so the data source can be swapped (real ↔ mock) without touching ViewModels. For flags, you can map FIFA/ISO country codes to a flag CDN (e.g. flagcdn.com) or bundle flag assets.

4. Testing tooling & admin panel

Because the tournament results unfold over weeks, you must be able to test the full loop today without waiting for real matches. Build a hidden **admin/mock control panel**:

- Inject **mock results** for upcoming games, then watch scores and the leaderboard update accordingly via the same automation path used in production.
- **Reset** controls: reset specific days, clear all guesses, clear/seed users, and a full reset. Keep each reset scoped and clearly labelled.
- Seed a handful of fake users with guesses so the leaderboard isn't empty during testing.

Access control

- The panel should be visible **only to you**. A hardcoded admin email (your future user) gating the panel is perfectly acceptable — keep the check in one place (e.g. an `isAdmin` getter) so it's easy to change.

5. Quality & cross-platform requirements

- Runs on **web, Android, and iOS**; responsive across phone, tablet and desktop widths with a consistent UX.
- **Unit tests** covering at least the scoring logic (single + set bonus), the ViewModels, and repository mapping. Aim for tests that would catch a regression in how points are awarded.
- Pleasant, clean UI — team flags on every game/guess row, sensible spacing, loading/empty/error states.
- Backed by **git** from the start (init early, commit in small logical steps). A short README explaining how to run it and how to open the admin panel.

- You're free to use **GitHub** and open a project/repository there for development. Still check with Shahar on the specifics (repo location, and whether the web build should be hosted) before you get too far in.
- English UI required; a **Hebrew toggle** (with RTL) is a nice-to-have, not a must. If you do it, use Flutter's localization so it's structured rather than hard-coded strings.
- Before you call it done, **run it on all three platforms** and exercise the mock panel end-to-end so we're not going back and forth on basic breakage.

6. Definition of done

- 1 App builds and runs on web, Android, and iOS.
- 2 Email sign up / sign in works and the session persists.
- 3 Upcoming games and past results load from a real data source, with flags shown.
- 4 Users can guess outcomes; guesses lock at kickoff.
- 5 Scoring (+1 per game, +2 set bonus) is correct and covered by unit tests.
- 6 Leaderboard shows the top 10.
- 7 Results/scores/leaderboard update automatically after a game finishes.
- 8 Admin-only mock panel can inject results and reset days/guesses/users.
- 9 Code is in git with a README; you've run the full flow yourself before handing it over.

7. Things you may not have thought about

A few edge cases and decisions worth calling out up front — flag your choices in the README:

- **Timezones.** Fixtures come in various UTC offsets. Decide on one display timezone (the user's local is reasonable) and store times in UTC. Kickoff/lock logic depends on this being right.
- **Guess locking.** Be explicit about when a guess can no longer be changed (we suggested kickoff). The admin panel shouldn't let you 'fix' a result before guesses are locked, or scoring gets confusing.
- **Draws and knockout stages.** Your three-way guess (win/win/draw) fits the group stage. Knockout games can't end in a draw after extra time/penalties — decide how a 'draw' guess resolves there, or scope the project to the group stage.
- **What counts as a 'set' for the +2 bonus.** We suggested 'all games on the same day', but some days have many games. Confirm the exact rule with your buddy and keep it in one configurable function.
- **Source of truth for results.** When the mock panel and the real API disagree (during testing), which wins? Make the data source explicitly switchable so there's no ambiguity.
- **Auth backend.** Firebase Authentication is the easy path here and handles password storage for you. If you go any other route, don't store plain-text passwords.
- **Ties on the leaderboard.** Define a tiebreaker (e.g. earliest to reach the score) so the top 10 is deterministic.

This is a learning project — if a requirement is ambiguous, make a reasonable call, write it down, and raise it. We care more about clean MVVM, tested scoring logic, and a working cross-platform build than about pixel perfection.